

Introduction

In this poster, we will:

- Justify the use of a Rotating Wave Approximation Hamiltonian
- Construct Quantum Logic Gates, starting with the Phase and Hadamard
- Look at an application of Quantum Logic Gates, the Quantum Fourier Transform

We want to introduce the Pauli matrices (Barnett 2026b, p. 15), [Physical/Exact Hamiltonian](#) and [Rotating Wave Approximation \(RWA\)](#) (Barnett 2026b, p. 18) to obtain quantum gates.

Pauli matrices:

σX = (0 1 / 1 0), σY = (0 -i / i 0), σZ = (1 0 / 0 -1)

Physical Hamiltonian (Barnett 2026b, p. 15):

H_exact(t) = (ε/2)σZ + γσX cos(ωt), ε, γ, ω ∈ ℝ≥0

In order to obtain the RWA, "it is customary to work entirely within the frame rotating at ω". (Barnett 2026b, p. 18)

Rotating Frame:

|ψ'(t)⟩ = e^{i(ε/2)σZ}|ψ(t)⟩, t ∈ ℝ≥0

ROT/RWA:

H_rot(t) = (ε-ω)/2 σZ + γ/2 σX + V(t), H_RWA = (ε-ω)/2 σZ + γ/2 σX

V(t) = γ/2 (σX cos(2ωt) - σY sin(2ωt))

We must use the Time-dependent Schrödinger Equation (Barnett 2026b, p. 9) to find a |ψ(t)⟩:

iħ ∂/∂t |ψ(t)⟩ = H(t)|ψ(t)⟩

ħ = 1 for all calculations in this poster.

Is the RWA viable?

Is the time-independent H_RWA solution a close approximation to the time-dependent H_rot(t) solution when we drop V(t)?

We obtain the following solutions to the Time-dependent Schrödinger Equation:

|ψ_RWA(t)⟩ = e^{-iH_RWA t}|ψ_RWA(0)⟩, |ψ_rot(t)⟩ = T exp(-i ∫_0^t H_rot(t') dt')|ψ_rot(0)⟩

for an inner product, with H being picked as the RWA and Rotated Hamiltonian for each function, T is the time-ordering operator, derived from a Dyson series (Cappellaro 2012, p. 4).

We define an inner product function on Python and compute the time-averaged state Fidelity (how close 2 Quantum states are to each other (Ivezic 2023)):

F̄ = 1/T ∫_0^T F(t) dt, F(t) = |⟨ψ_rot(t)|ψ_RWA(t)⟩|^2, 0 ≤ F̄ ≤ 1, T ∈ ℝ≥0

Deriving F(t) from Born Rule (Barnett 2026a, p. 11), the F̄ is calculated for T = 10 using the Trapezium rule, using 10,000 points and a NumPy array (Max. Fidelity Error = 1 - min(Fidelity)).

Table 1: Fidelity and Max Fidelity Error (M.F.E). Obtained using NumPy (Harris et al. 2020) and SciPy (Virtanen et al. 2020) (trapezoid function).

ε	γ	ω	Fidelity	Max. Fidelity Error
11	1	10	0.9980863	0.0066193
41	1	40	0.9998697	0.0004489
1001	1	1000	0.9999995	1.245E-6

When √(ε - ω)² + γ² ≪ ω is obeyed (which is required for negligible oscillations (Barnett 2026b, p. 17)), the solutions to the Schrödinger Equation are extremely similar, so we can drop V(t).

Realised Single Qubit Gates

Using our Rotating Wave Approximation, we must obtain realised gates of the form U(τ) = e^{-iH_RWA τ} (Barnett 2026b, p. 20) and compare U to the sought form of a quantum gate. What parameters do we substitute in order to realise the sought quantum gates?

Gate	Sought Form	γ	τ	ε
Phase	e^{-iφσZ}	0	2φ / (ε-ω)	≪ ω
Hadamard	1/√2 (σX + σZ)	ε - ω	π / (√2 γ)	≪ ω
X/NOT	σX	≠ 0	(2k+1)π / γ	ω
√X	√σX	≠ 0	(2k+1)π / 2γ	ω
π/8/T	e^{-iσZ π/4}	0	π / (2(ε-ω))	≪ ω
S	e^{-iσZ π/2}	0	π / (ε-ω)	≪ ω
RX(θ)	e^{-iσX θ/2}	≠ 0	θ / γ	ω
RY(θ)	e^{-iσY θ/2}	≠ 0	θ / γ	ω

These are correct up to global phase, which is physically irrelevant.

RX(θ), RY(θ) (derived via identity) are the Rotation gates with a respective axis X and Y. θ ∈ ℝ

U(θ, φ, λ) is the General Single Qubit Rotation Gate defined as RZ(φ)RY(θ)RZ(λ) (φ, θ, λ ∈ ℝ). Can be derived using the ideas of rotations. (Nielsen and Chuang 2010, p. 174-177)

Picking φ = π/2 in the Phase gate yields RZ(θ).

k ∈ ℤ for the X/NOT and √X gates.

It is also very important to know the Dyadic Rational Phase Gate (Nielsen and Chuang 2010, p. 218):

Rk = (1 0 / 0 e^{2πi/2^k})

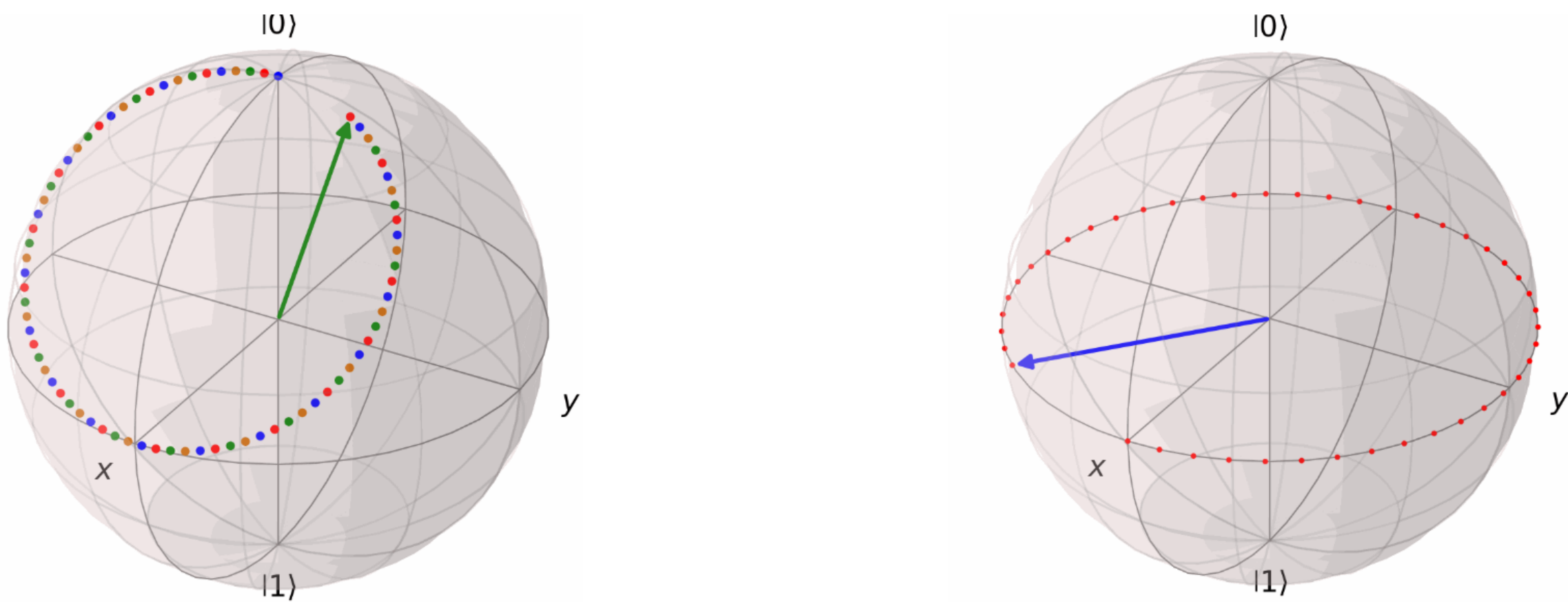
We later use this gate for Quantum Fourier Transforms, with R1 = Z, R2 = S, R3 = π/8/T gate. (k ∈ ℕ)

The Bloch Sphere

It is helpful to be able to visualise these quantum gates, and a useful way of doing this is the Bloch Sphere. To get the idea:

- σX, σY and σZ are π radians anticlockwise rotations, up to a global phase, with exponentiation causing continuous rotation (e.g. RX(π) ≡ X as RX(π) = -iX).
- The Phase gate is an anticlockwise rotation about the Z axis, angle φ.

Below, the Hadamard gate has been applied twice on |0⟩ and the Phase gate has been applied on |+⟩ = 1/√2(|0⟩ + |1⟩). It can be seen below that H² = I.



These are produced as GIFs using Python's QuTip (Lambert et al. 2026), NumPy (Harris et al. 2020), ImageIO (ImageIO Contributors 2026) and OS packages.

Quantum Fourier Transform

Now we want to introduce the 2-qubit SWAP gate by name, which will be important in the application of the [Quantum Fourier Transform \(QFT\)](#).

We define the QFT as the Quantum analogue of the Discrete Fourier Transform. The QFT (Nielsen and Chuang 2010, p. 217) definition is seen below:

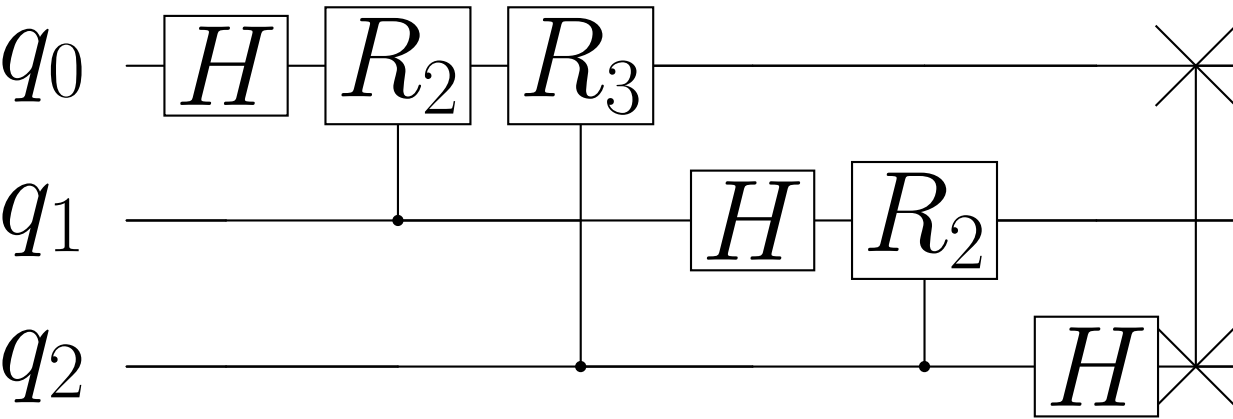
|j⟩ ↦ 1/√N ∑_{k=0}^{N-1} e^{2πijk/N} |k⟩

Where N: Length of state, |k⟩: basis states that appears in transformed data, |j⟩: a complex state of input data. When computing a Quantum Fourier Transform, we want to take N = 2^n where n is the number of qubits in a Quantum Computer.

Writing |j⟩ = |j1, ..., jn⟩ in a binary representation, where j = j1j2...jn, we can eventually derive a product expression (Nielsen and Chuang 2010, p. 218) for the QFT using tensor operations:

|j⟩ ↦ (|0⟩ + e^{2πi0.jn}|1⟩)(|0⟩ + e^{2πi0.j_{n-1}.j_n}|1⟩)...(|0⟩ + e^{2πi0.j1.j2...j_n}) / 2^{n/2}

Where we use the notation 0.j1j2...jn = j1/2 + j2/4 + ... + jn/2^{n-1} which represents a binary fraction. To see what this visually looks like for n = 3 (a 3-qubit QFT), we present the following circuit:



Note that the initial state is represented as |q0q1q2⟩. This figure was created using Qiskit (Javadi-Abhari et al. 2024) and verified using NVIDIA's CUDA-Q v13.2 (NVIDIA 2026), applying a comparison against the QFT matrix that we apply on |000⟩ to obtain our expected result (Qiskit and CUDA-Q are Python packages).

We can see the SWAP gate being used at the end of the QFT, where at most n/2 SWAP gates are required for a given QFT applied on n qubits (Nielsen and Chuang 2010, p. 220).

We can compare the QFT (on n qubits) to the classical algorithm, the Fast Fourier Transform (FFT) (on N samples) (Nielsen and Chuang 2010, p. 220):

[QFT, Θ(n²)] [FFT, Θ(N log₂ N)], N = 2^n

From this we can see that the QFT requires Θ(n²) quantum gates and FFT requires Θ(N log₂ N) classical operations. Under ideal conditions, the QFT is more efficient than the FFT.

If I had more time, I would consider the following:

- Shor's Algorithm, O((log(N))³), for factorising integers (Nielsen and Chuang 2010, p. 37).
- Grover's Algorithm, O(√N), for unordered searches of N elements.
- Limitations of the QFT (hardware, availability)

As per college guidelines, I acknowledge that I have used GenAI for the purpose of searching for literature (textbooks, Python packages), solving LaTeX issues (formatting, packages) and getting feedback on my work (including GitHub files).

Citations, Derivations and Python

